

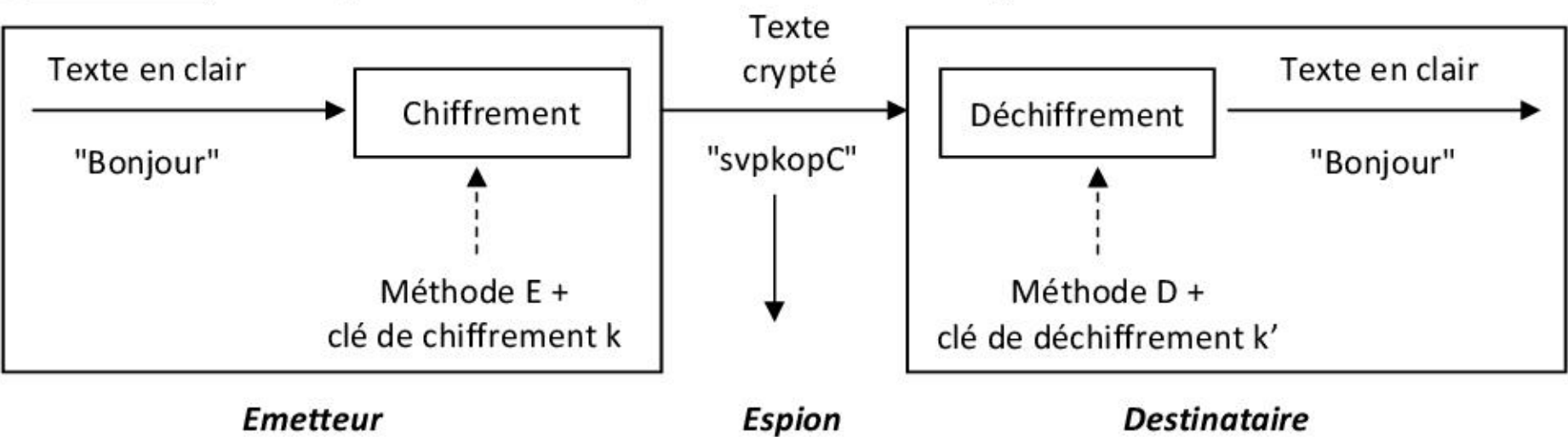
Cryptographie : Outil indispensable de la sécurité des échanges

Sommaire

I.	Principe :	1
II.	Usages :	2
1.	Confidentialité	2
2.	Intégrité	2
3.	Authentification	2
III.	Fonctionnement et vocabulaire :	2
1.	Hachage	2
2.	Salage	3
3.	Chiffrement	4
4.	Déchiffrement	4
5.	Chiffrement symétrique – A clé secrète	4
6.	Chiffrement asymétrique – A clé publique	5
7.	Man In The Middle	Erreur ! Signet non défini.
8.	Force brute (attaque par)	Erreur ! Signet non défini.
9.	Cryptanalyse	Erreur ! Signet non défini.
IV.	Certificat de sécurité	Erreur ! Signet non défini.
1.	Fonction	Erreur ! Signet non défini.
2.	Contenu	Erreur ! Signet non défini.
3.	Processus de signature d'un document	Erreur ! Signet non défini.
4.	Organisation	Erreur ! Signet non défini.
5.	Mécanisme de diffusion des certificats	Erreur ! Signet non défini.

I. Principe :

Faire transiter un texte lisible (en clair, à gauche) d'un émetteur vers un destinataire, en le rendant le plus incompréhensible temporairement, mais en pouvant le décoder pour retrouver le message initial :



Ici, la méthode de chiffrement E (encryptage) consiste à *renverser les lettres du mot et à les décaler dans l'alphabet de ... 1 lettre* (la clé de chiffrement)

La méthode de déchiffrement D (decryptage) est identique, mais utilise une clé différente (*-1 lettre*). Ou vous considérez que l'algorithme est différent et la clé identique...

II. Usages :

1. Confidentialité

Les données échangées qui peuvent être interceptées sont cryptées, donc illisibles. Leur contenu n'est donc pas révélé, sauf si l'espion dispose de la méthode de déchiffrement et de la clé associée.

2. Intégrité

Si l'espion ne connaît pas la méthode de chiffrement et/ou la clé utilisée, il ne pourra pas générer ses propres données, mais il pourra modifier le message crypté cependant, de manière cohérente ou non. Il en est tout autrement s'il connaît la méthode de chiffrement et sa clé (il pourra remplacer les données reçues par les siennes..).

3. Authentification

Il n'y a pas d'authentification entre émetteur et récepteur à ce stade (le destinataire ne peut pas être certain que c'est bien l'émetteur supposé qui est à l'origine de l'échange, et réciproquement) : Elle est nécessaire pour sécuriser l'échange.

III. Fonctionnement et vocabulaire :

1. Hachage

A ne pas confondre avec le chiffrement, le hachage produit une **empreinte numérique**, soit une valeur de longueur fixe selon l'algorithme utilisé (ex MD-5, SHA-1, SHA-2-256, ...).

Il est utilisé dans deux cas de figure bien distincts : Intégrité des données pendant un transfert/stockage, et authentification d'un utilisateur par exemple.

Le principe est le suivant : Un hash est calculé sur une donnée d'origine. Au moment de la vérification (réception de données ou bien identification), le hash est recalculé avec la même fonction de hachage sur les données fournies. S'il produit le même résultat, on considère que les données n'ont pas été altérées pendant le transfert/stockage, ou que la personne s'est bien authentifiée.

Cependant, des « collisions » existent (deux valeurs différentes produisant le même hash). Tout simplement parce que les valeurs soumises à la fonction de hachage sont infinies, tandis que la fonction de hachage délivre un résultat d'une longueur fixe, donc avec un nombre fini de possibilités.

Algorithme	Longueur de l'empreinte (hash)
MD-5	128 bits, 16 octets, 32 chiffres hexa
SHA-1	160 bits, 20 octets, 40 chiffres hexa
SHA-2-256	256 bits, 32 octets, 64 chiffres hexa

Exemple :

Calculer un hash basé sur le total des octets. Les fonctions produisant un résultat avec un format donné, on limite le total à 32 bits ($2^{32} = 4\,294\,967\,296$)

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1																				
2		Données																		Hash
3																				
4	Message :	V	i	r	e	r		1	0	0	€		à		A	I	i	c	e	
5	Octets envoyés :	86	105	114	101	114	32	49	48	48	128	32	224	32	65	108	105	99	101	1591
6																				
7																				
8	Octets reçus	86	105	114	101	114	32	57	48	48	128	32	224	32	65	108	105	99	101	1599
9	Message :	V	i	r	e	r		9	0	0	€		à		A	I	i	c	e	
10																				
11																				
12	Octets reçus	86	105	114	101	114	32	49	48	48	128	32	224	32	67	101	108	105	97	1591
13	Message :	V	i	r	e	r		1	0	0	€		à		C	e	l	i	a	
14																				

L'émetteur envoie "Virer 100€ à Alice" (en réalité les octets en dessous dans le jeu de caractères)

Le hash calculé vaut $86+105+114+\dots+99+101 = 1591$

1^{ère} hypothèse : le récepteur reçoit les octets donnant un hash = 1599

En le comparant au hash du message initial 1591, il détectera sa perte d'intégrité et donc refusera "Virer 900€ à Alice"

2^{ème} hypothèse : le récepteur reçoit les octets donnant un hash = 1591

Le hash étant identique, le récepteur considère le message comme intègre, ce qui n'est pas le cas. Ceci est dû à la collision de la fonction de hachage, qui est de très mauvaise qualité.

Fonctions de hachage obsolètes : MD5, SHA-0, SHA-1

Fonctions de hachage conseillées : SHA-2 et SHA-3, produisant des hash sur 224, 256, 384 et 512 bits.

Avec l'exemple précédent, cela donnerait :

Fonction de hachage	Virer 100€ à Alice	Virer 100€ à Celia
MD5	eb859ae2afcf30502dfe57965e3d556a	d544f09b8fad29f34be8cc828f78f679
SHA-2-256	7505904d8391d62668af9bddb57edc412cf0fc02d cc3f457c6ee5f85955acced	546d73e919bd90de2e4a82a38567af9493cac3866f7893 a1f0562860bdbed61d

Utilisation classique : On ne stocke pas les mots de passe en clair dans les bases de données, car l'accès à la base compromettrait toute la sécurité.

On préfère conserver **en base de données les hachages des mots de passe**. La proposition de mot de passe est elle aussi hachée (de la même manière), et si elle correspond au hash stocké, il en est conclu (peut-être à tort cf *collisions*) la bonne identification.

Le hachage participe à l'intégrité des données.

2. Salage

Si le hachage ne porte pas uniquement sur la valeur proposée (par exemple le mot de passe), et intègre une donnée complémentaire qui peut varier (par exemple le nom du compte de l'utilisateur), le hachage est réputé plus compliqué à décoder (craquer), ceci s'appelle le salage du mot de passe.

Cas d'utilisation : On conserve en base de données, pour chaque utilisateur, son compte (pseudo) et mot de passe :
Compte (pseudo, mdp)

1^{er} cas : mot de passe en clair.

Exemple de contenu : ("admin", "secret"), ("bob", "123456"), ("julie", "secret"), ...

Si le hacker a accès à la table, c'est gagné pour lui.

2^{ème} cas : mot de passe haché (avec une fonction de hachage volontairement trop faible pour illustrer).

Exemple de contenu : ("admin", "HOWui="), ("bob", "URSwy="), ("julie", "HOWui="), ...

HOWui= est obtenu par : *fonctionDeHachage* ("secret")

Pour que le hacker puisse obtenir le mot de passe du compte *admin*, il suffit qu'il connaisse le mot de passe d'un autre compte produisant le même hash (car ce seront les hash qui seront comparés pour l'identification...), ou qu'il utilise les tables arc-en-ciel (*Rainbow tables*) vues ci-après. Il pourra ici se connecter avec *admin/secret*

On trouve sur le net des « rainbow tables » (tables arc-en-ciel), donnant pour un hash donné et une fonction de hachage donnée, une valeur produisant ce hash. Dans ce cas, proposer cette valeur comme mot de passe fonctionne, puisque seul le mot de passe détermine le hash.

3^{ème} cas : mot de passe haché+salé avec le nom du compte (fonction de hachage volontairement trop faible pour illustrer).

Exemple de contenu : ("admin", "LKUHg="), ("bob", "BFgk="), ("julie", "DFhxn="), ...

LKUHg= est obtenu par : *fonctionDeHachageSalage* ("admin", "secret")

Le hacker ne peut plus s'appuyer sur la connaissance du mot de passe d'un autre compte, car les hashes identiques ne permettent pas de déduire le mot de passe.

Il ne peut pas non plus s'appuyer sur les « rainbow tables » puisque la proposition de mot de passe qu'il y trouvera éventuellement sera salé avant d'être hachée.

Plus de détails sur : <https://www.ionos.fr/digitalguide/serveur/securite/rainbow-tables/>

Le salage renforce le contrôle de l'intégrité des données.

3. Chiffrement

C'est l'action de codifier (encoder) le message en clair au moyen d'un algorithme (une méthode), lequel a besoin d'un élément confidentiel (la clé de chiffrement). Il produit un texte illisible pour qui ne possède pas la méthode de déchiffrement et la clé associée.

Le chiffrement assure la confidentialité des données.

4. Déchiffrement

Basé sur un principe similaire au chiffrement, il utilise une méthode et une clé lui aussi. Si la méthode et la clé de déchiffrement sont bien associées à la méthode et clé de chiffrement, le résultat redonne le message initial.

5. Chiffrement symétrique – A clé secrète

Si la méthode de chiffrement est la même que celle pour le déchiffrement, les clés associées sont identiques, et l'on parle dans ce cas de chiffrement symétrique. La clé est dite secrète car il ne faut pas la dévoiler pour garder les données confidentielles pendant le transport/stockage.

Démo : Application *Etapas Chiffrement Ransomware*

Exemple : Augmenter de 1 (modulo 256) –ou une autre valeur - chaque octet envoyé, et retirer 1 – ou l'autre valeur - à chaque octet reçu :

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1																			
2		Données																	
3																			
4	Message :	V	i	r	e	r		1	0	0	€		à		A	I	i	c	e
5	Octets :	86	105	114	101	114	32	49	48	48	128	32	224	32	65	108	105	99	101
6	Octets encodés :	87	106	115	102	115	33	50	49	49	129	33	225	33	66	109	106	100	102
7	Message envoyé :	W	j	s	f	s	!	2	1	1		!	á	!	B	m	j	d	f
8																			
9																			
10	Octets reçus	87	106	115	102	115	33	50	49	49	129	33	225	33	66	109	106	100	102
11	Message reçu :	W	j	s	f	s	!	2	1	1		!	á	!	B	m	j	d	f
12	Octets décodés :	86	105	114	101	114	32	49	48	48	128	32	224	32	65	108	105	99	101
13	Message :	V	i	r	e	r		1	0	0	€		à		A	I	i	c	e
14																			

La méthode de chiffrement/déchiffrement est l'algorithme utilisé (ici ajout/retrait modulo 256), et la clé est la valeur utilisée (ici 1).

Pour assurer la confidentialité et l'intégrité de l'échange, il faut ici **assurer le secret partagé** (entre émetteur et récepteur) de la méthode et de la clé commune utilisée.

6. Chiffrement asymétrique – A clé publique

Dans le cas contraire, si la méthode de chiffrement est totalement différente de celle pour déchiffrer, les clés associées sont différentes aussi, et l'on parle dans ce cas de chiffrement asymétrique.

RSA est le nom de l'algorithme utilisé très souvent dans cette situation.

Dans ce cas, **l'une des clés est dite privée**, car détenue par une seule personne (l'émetteur) tandis que **l'autre est dite publique** car diffusée à toute personne susceptible d'échanger avec l'émetteur.

Ce qui est crypté par une clé (privée ou publique) ne peut être décrypté que par l'autre clé.

Illustration avec le cas classique entre un serveur web sécurisé (HTTPS) et un navigateur :

